

Direct device access from the SmartNIC towards datacenter disaggregation

Master's thesis meeting : week 2

Nicolas Jeanmenne

Table of contents

- Retrieve bdev info
 - Setup
 - Descriptor and stuff
 - Retrieve info
- Demo #1 !
- Write and read a string
 - Buffer allocation
 - Write process
 - Read process
- Demo #2 !
- Write and read a file (cp)
 - Modifications in main
 - Modification for buffer alloc
- Demo #3 !
- Conclusion
- TODOs for week 3

Retrieve bdev info

- Get name, product, block, ... of bdev
- No I/O yet
- File : `src/get_info.c`

Setup

- Opts and context struct
- Attach a bdev from `simple-config.json`
- Start discover function

```
int main(int argc, char **argv) {
    struct spdk_app_opts opts = {};
    struct discover_ctx_t discover_ctx = {};
    int rc;

    spdk_log_set_print_level(SPDK_LOG_DEBUG);
    spdk_log_set_level(SPDK_LOG_DEBUG);

    spdk_app_opts_init(&opts, sizeof(opts));
    opts.name = "bdev_name";
    opts.json_config_file = "./simple-config.json";
    opts.env_context = "-a 0000:02:00.0";

    rc = spdk_app_start(&opts, discover, &discover_ctx);

    if (rc) {
        SPDK_ERRLOG("Error starting spdk (rc=%d)\n", rc);
    }

    SPDK_NOTICELOG("Successfully finished !\n");
    spdk_app_fini();
    return rc;
}
```

Descriptor and stuff

- Take first bdev available
- Open descriptor in order to get info
- Open io channel

```
24 static void discover(void *arg1) {
25
26 @ SPDK_NOTICELOG("Entering discover function\n");
27 @ struct discover_ctx_t *discover_ctx = arg1;
28 @ struct spdk_bdev *bdev;
29 @ discover_ctx->bdev = NULL;
30 @ discover_ctx->bdev_desc = NULL;
31 @ int rc;
32 @
33 @ bdev = spdk_bdev_first();
34 @
35 @ if (bdev == NULL) {
36 @ @ SPDK_ERRLOG("Ca bdev pas\n");
37 @ @ spdk_app_stop(-1);
38 @ @ return;
39 @ }
40 @ discover_ctx->bdev_name = spdk_bdev_get_name(bdev);
41 @
42 @ rc = spdk_bdev_open_ext(discover_ctx->bdev_name, false, callback, NULL,
43 @ @ @ @ @ @ @ @ @ @ @ @ @ @ @ @ &discover_ctx->bdev_desc);
44 @ if (rc) {
45 @ @ SPDK_ERRLOG("Descriptor error : %d", rc);
46 @ @ spdk_app_stop(-1);
47 @ @ return;
48 @ }
49 @
50 @ discover_ctx->bdev = spdk_bdev_desc_get_bdev(discover_ctx->bdev_desc);
51 @ discover_ctx->bdev_io_channel =
52 @ @ @ spdk_bdev_get_io_channel(discover_ctx->bdev_desc);
53 @ if (discover_ctx->bdev_io_channel == NULL) {
54 @ @ SPDK_ERRLOG("No io channel for : %s", discover_ctx->bdev_name);
55 @ @ spdk_bdev_close(discover_ctx->bdev_desc);
56 @ @ spdk_app_stop(-1);
57 @ @ return;
58 @ }
59 @ }
```

Retrieve info

```
17 @  
18 @ const char *name = spdk_bdev_get_name(discover_ctx->bdev);  
19 @ const char *product = spdk_bdev_get_product_name(discover_ctx->bdev);  
20 @ uint32_t bsize = spdk_bdev_get_block_size(discover_ctx->bdev);  
21 @ uint64_t iotime = spdk_bdev_get_io_time(discover_ctx->bdev);  
22 @
```

Demo #1 !

Write and read a string

- Start from previous code
- Write a *(fixed)* string into a DMA buffer
- Reset the same buffer
- Read the content of bdev into the latter
- File : `src/rwstring.c`

Buffer allocation

- Set up a buffer to print the string into
- Go to `write_string`
- If zoned bdev : reset zone and go to `write_string`

```
26 @ SPDK_NOTICELOG("DMA alloc for write\n");
25 @ uint32_t buf_align;
24 @ discover_ctx->buf_size = spdk_bdev_get_block_size(discover_ctx->bdev) *
23 @ 0 0 0 0 0 0 0 0 0 0 0 0 spdk_bdev_get_write_unit_size(discover_ctx->bdev);
22 @ buf_align = spdk_bdev_get_buf_align(discover_ctx->bdev);
21 @ discover_ctx->buf = spdk_dma_zmalloc(discover_ctx->buf_size, buf_align, NULL);
20 @
19 @ if (!discover_ctx->buf) {
18 @ @ SPDK_ERRLOG("DMA alloc failed :(\n");
17 @ @ spdk_put_io_channel(discover_ctx->bdev_io_channel);
16 @ @ spdk_bdev_close(discover_ctx->bdev_desc);
15 @ @ spdk_app_stop(-1);
14 @ }
13 @
12 @ snprintf(discover_ctx->buf, discover_ctx->buf_size, "%s",
11 @ @ 0 0 0 0 0 "Maybe you should use a real docs ? \n");
10 @ @ 0 0 0 0 0
9 @ if (spdk_bdev_is_zoned(discover_ctx->bdev)) {
8 @ @ reset_zone(discover_ctx);
7 @ @ return;
6 @ }
5 @ write_string(discover_ctx);
4 }
```

Write process

- Queuing write request if no memory available
- Callback function on completion

[illegible]

Read process

- Buffer content zeroed in write callback
- Then same idea but with read call

[illegible]

Demo #2 !

Write and read a file (cp)

- Same idea but with an arbitrary file
- File : `src/basic-cp.c`

Modifications in main

- Parse arg for file input (from linux fs)
- Get the file stat

```
int main(int argc, char **argv) {
    struct spdk_app_opts opts = {};
    struct context_t context = {};
    int rc;

    spdk_log_set_print_level(SPDK_LOG_DEBUG);
    spdk_log_set_level(SPDK_LOG_DEBUG);

    spdk_app_opts_init(&opts, sizeof(opts));
    opts.name = "bdev_name";
    opts.json_config_file = "./simple-config.json";
    opts.env_context = "-a 0000:02:00.0";

    rc = spdk_app_parse_args(argc, argv, &opts, "f:o:", NULL, parse_arg, usage);
    if (rc == SPDK_APP_PARSE_ARGS_FAIL) {
        SPDK_ERRLOG("Error parsing args (invalid args) : %d\n", rc);
        exit(rc);
    }

    if (g_opts.input_path == NULL) {
        SPDK_ERRLOG("Missing args\n");
        usage();
        rc = EINVAL;
        exit(rc);
    }

    g_opts.fd = open(g_opts.input_path, O_RDONLY);
    if (g_opts.fd == -1) {
        SPDK_ERRLOG("Failed to open %s\n", g_opts.input_path);
        exit(-1);
    }

    if (fstat(g_opts.fd, &g_opts.fstat) == -1) {
        SPDK_ERRLOG("Failed to read file stat of : %s\n", g_opts.input_path);
        exit(-1);
    }
}
```

Modifcation for buffer alloc

- Allocation must be a multiple of `block_size`
- Put content into the buffer (thus involving host CPU for this step)
- Same write and read approach as before

Direct device access from the SmartNIC towards datacenter disaggregation (Nicolas Jeanmenne)

```

size_t fsize = g_opts.fstat.st_size;
uint32_t block_size = spdk_bdev_get_block_size(ctx->bdev);
uint64_t num_blocks = (fsize + (block_size - 1)) / block_size;

SPDK_NOTICELOG("DMA alloc for write\n");
ctx->buf_size =
    (num_blocks * block_size) * spdk_bdev_get_write_unit_size(ctx->bdev);
SPDK_NOTICELOG("fsize : %lu\nbuf_size : %d\n", fsize, ctx->buf_size);
uint32_t buf_align = spdk_bdev_get_buf_align(ctx->bdev);
SPDK_NOTICELOG("Buf align : %d\n", buf_align);
ctx->buf = spdk_dma_zmalloc(ctx->buf_size, buf_align, NULL);

if (!ctx->buf) {
    SPDK_ERRLOG("DMA alloc failed :(\n");
    spdk_put_io_channel(ctx->bdev_io_channel);
    spdk_bdev_close(ctx->bdev_desc);
    spdk_app_stop(-1);
}

int r = read(g_opts.fd, ctx->buf, fsize);
if (r == -1) {
    SPDK_ERRLOG("Failed to copy file in DMA buffer");
    spdk_put_io_channel(ctx->bdev_io_channel);
    spdk_bdev_close(ctx->bdev_desc);
    spdk_app_stop(-1);
}

SPDK_NOTICELOG("I/O : writing to bdev : %s\n", ctx->bdev_name);
rc = spdk_bdev_write(ctx->bdev_desc, ctx->bdev_io_channel, ctx->buf, 0,
    ctx->buf_size, write_complete, ctx);
if (rc == -ENOMEM) {
    SPDK_NOTICELOG("Queuing I/O\n");
    ctx->bdev_io_wait.bdev = ctx->bdev;
    ctx->bdev_io_wait.cb_fn = write_file;
    ctx->bdev_io_wait.cb_arg = ctx;
    spdk_bdev_queue_io_wait(ctx->bdev, ctx->bdev_io_channel,
        &ctx->bdev_io_wait);
} else if (rc) {
    SPDK_ERRLOG("%d error while writing to bdev: %s\n", spdk_strerror(-rc), rc);
    spdk_put_io_channel(ctx->bdev_io_channel);
    spdk_bdev_close(ctx->bdev_desc);
    spdk_app_stop(-1);
}
}

```

15

Demo #3 !

Conclusion

- We can copy a file !
- Writing SPDK code is quite fun
- But non-negligible time taken to understand what function does what

TODOs for week 3

- Add a flag to retrieve file from bdev
- Performances evaluation
- ...

That's all for today !

Direct device access from the SmartNIC towards datacenter
disaggregation (Nicolas Jeanmenne)

